Proceedings of the 2005 IEEE
International Conference on Robotics and Automation
Barcelona, Spain, April 2005

Deadlock Avoidance Petri Net Controller for Manufacturing Systems with Multiple Resource Service

Keli Xing Xiajie Jin Yu Feng

School of Mechanical and Electronic Engineering and Automation,
Shanghai University, 200072, Shanghai,China
xingkelii@163.com

Abstract: **Many important Petri net-based methods have been presented to prevent deadlocks in automated manufacturing system with concurrent sequential processes. This paper addresses deadlock problems in automated manufacturing systems with multiple resource service. Petri net is used to model the flow of the parts and the usage and release of the resources. Deadlock structure objects in Petri net models, which lead the system to deadlock, are characterized. This paper proves that, by adding a control place for each deadlock structure so that the number of parts in the deadlock structure is limited, deadlock can be successfully prevented. Using the including relation between deadlock structures, from inside to outside, the Petri net controller is synthesized, and it is proved that the presented controller is a maximally primitive Petri net controller.**

Keywords: **Automatic Manufacturing Systems, Petri Net, Deadlock Avoidance, Optimal Control**

I. INTRODUCTION

Many works devoted to deadlock avoidance problems in the automated manufacturing systems with sequential processes in which every operation is done by only one resource. In such a system all operations of a part are executed in a predefined order and every operation needs a predefined resource [1, 2, 6, 8]. Recently some works propose deadlock avoidance strategies for assembly/disassembly systems [4, 7] and for the systems with multiple resource acquisition [3, 5]. They use Petri net to model the systems and derive deadlock avoidance algorithms.

This paper considers automated manufacturing systems with multiple resource service, that is, the process of each part is a predefined sequence of operations, and some operation requires more than one kind of resources, but at most one unit of each type of resources is used by an operation. We use Petri nets to model the systems and characterize the necessary and sufficient conditions for deadlock by Petri net substructures called deadlock structures. This paper proves that by adding a control place for each deadlock structure to limit usage of resources in it, deadlock can be successfully prevented and the presented controller is optimal.

The paper is organized as follows. Section 2 introduces the notations of Petri nets and the system description and builds the Petri net model of the systems. Section 3 analyzes deadlocks in the Petri net models and characterizes Petri net substructures that lead the system to deadlock. Section 4 defines the deadlock avoidance Petri net controllers. And finally Section 5 is conclusion.

II. PETRI NET MODELING THE MANUFACTURING SYSTEMS WITH MULTIPLE RESOURCE SERVICES

In this section we describe the automated manufacturing systems with Multiple Resource Services, and model the system with Petri Net.

A. Description of Automated Manufacturing Systems

The automated manufacturing system considered in this paper consists of m types of resources. Let r_i, i = 1, 2, …, m, denote the resource types of the system. The capacity of r_i is an integer, say C_{ri}, indicating the maximum number of parts that such a resource can simultaneously hold. The system can process n types of parts. The processing procedure of a part is a sequence of operations. Each operation requires some resources, but at most one unit of one kind of resources. So the processing procedure of a q-type part can represented by $PR(q) = (R_1, R_2, \ldots, R_k)$ where k is the number of operations necessary to complete the part and R_i is the set of resource types required by the ith operation. Here we suppose that $R_i \neq R_{i+1}$, i = 1, 2, …, k-1; Otherwise the ith and (i+1)th operations can be considered as the same.

B. Overview of Petri Nets

Throughout this paper, the following notations of Petri nets are used.

A Petri net is a 3-tuple PN = (P, T, F) where P is the finite set of places and T is the finite set of transitions. $F \subseteq (P\times T) \cup (T\times P)$ is called the flow relation or the set of directed arcs.

Let PN = (P, T, F) be a Petri net. The preset of a node $x \in P \cup T$ is defined as $\bullet x = \{y \in P \cup T \mid (y, x) \in F\}$. The postset of a node $x \in P \cup T$ is defined as $x\bullet = \{y \in P \cup T \mid (x, y) \in F\}$. The preset (postset) of a node set is defined as the union of the presets (postsets) of its all nodes.

The state or marking of a PN is a mapping $M: P \rightarrow N_+$, where N_+ is the set of nonnegative integers, and M is described by a |P|-vector and M(p) represents the number of tokens in the place $p \in P$ at the marking M. A transition $t \in T$ is enabled at a marking M iff for each place $p \in \bullet t$, M(p)>0. When t is enabled it can fire. The firing of t leads the system to a new marking M' where $M'(p) = M(p)-1$ if $p \in \bullet t \backslash t\bullet$; $M'(p) = M(p)+1$ if $p \in t\bullet \backslash \bullet t$; and otherwise $M'(p) = M(p)$. Let $RM(PN, M_0)$ denote the set of all reachable markings from the marking M_0.

C. Petri Net Modeling Automated Manufacturing Systems

Let us now develop the PN model of the automated manufacturing systems. First, we model the processing procedure of a type of parts by a path of transitions and

0-7803-8914-X/05/$20.00 ©2005 IEEE.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on March 08,2026 at 05:17:56 UTC from IEEE Xplore. Restrictions apply.

places. Each place represents an operation, called an operation place, and firing of each transition corresponds to one operation completing and the next operation starting. A token in a path represents a part as it flows through the system. For example, if the processing procedure of a type-q part is $PR(q) = (R_1, R_2, \ldots, R_k)$, then the Petri net model of its processing procedure is $t_1p_1t_1p_2t_2\ldots p_kt_k$ where p_i is an operation place, t_i is a transition. The firing of t_1 represents that a type-q part enters the system and begins its processing procedure. And the firing of t_k represents the finishing of its all operations and does not require any resources. Let $RE(p_i)$ denote the set of all resources that the operation p_i requires. Then $RE(p_i) = R_i$, $i = 1, 2, \ldots, k$. Let P and T denote the sets of all operation places and all transitions in all directed paths, respectively. The initial markings of operation places are 0, that is, for any place $p \in P$, $M_0(p) = 0$.

For each type of resource r, we assign a place, called a resource place and denoted also by r, for simplicity. Tokens in the place r indicate available resources of type-r. The initial marking is $M_0(r) = C_r$. Let R denote the set of all resource places.

To model the usage and release of resources, add an arc from the resource place r to transition t_i if $r \notin RE(p_i)$ and $r \in RE(p_{i+1})$ and an arc from t_i to r if $r \in RE(p_i)$ and $r \notin RE(p_{i+1})$.

Then the whole Petri net model of the system is $PN = (P \cup R, T, F, M_0)$, called a system Petri net model.

Given a transition $t \in T$, let ${}^{(p)}t$ and $t^{(p)}$ denote the input operation place and the output operation place of t, respectively, and let ${}^{(r)}t$ and $t^{(r)}$ denote the set of input resource places and the set of output resource places of t, respectively. For a given marking $M \in RM(PN, M_0)$, if $M({}^{(p)}t)>0$, t is called operation-enabled. If for any resource $r \in {}^{(r)}t$, $M(r)>0$, t is called resource-enabled. In the Petri net PN, only transitions resource- and operation-enabled at the same time can be fired.

For a given resource $r \in R$, let H(r) denote the set of all operation places that require the resource r, that is, $H(r) = \{p \in P \mid r \in RE(p)\}$. Let P_1 be a set of operation places and M a marking of PN, denote $M(P_1, r) = \sum_{p \in P_1, r \in RE(p)} M(p)$ and $M(P_1) = \sum_{p \in P_1} M(p)$.

D. Petri Net Controller

To deal with deadlock problems in the systems, a control place and some related arcs are added to the Petri net model for every structure object that leads the system to deadlock. That is, one control place is used to avoid one deadlock occurring.

III. DEADLOCK ANALYSES OF SYSTEM PETRI NET MODELS

In this section we use system Petri net models to analyze deadlock problems in automated manufacturing systems, and establish the necessary and sufficient conditions for deadlock in the systems. And the definition of deadlock is as follows.

Definition 1: Given a system Petri net model $PN = (P \cup R, T, F, M_0)$, a marking $M \in RM(PN, M_0)$, and a transition $t \in T$, if for any marking $M' \in RM(PN, M)$, t is not enabled at the marking M', then t is said to be in deadlock in the marking M. Let T_1 be a set of transitions. If any transition of T_1 is in deadlock in the marking M, T_1 is said to be in deadlock in the marking M.

Given a marking M of PN, we are interested in transitions which is in deadlock and also operation-enabled in the marking M. Let D(M) denote the set of all transitions which are in deadlock and operation-enabled in the marking M.

From the deadlock definition and the system Petri net models, we can prove the following sufficient condition for leading systems to deadlock.

Lemma 1: Given a system Petri net model $PN = (P \cup R, T, F, M_0)$ and a marking $M \in RM(PN, M_0)$, if a set of transitions T_1 and a set of resources R_1 satisfy the following conditions:

(1) For any transition $t \in T_1$, $M({}^{(p)}t)>0$;
(2) For any resource $r \in R_1$, $M({}^{(p)}T_1, r) = C_r$;
(3) For any transition $t \in T_1$, ${}^{(r)}t \cap R_1 \neq \varnothing$,

then $T_1 \subseteq D(M)$ is in deadlock in the marking M.

Proof: The condition (1) implies that every transition in T_1 is operation-enabled. From (3), we can see that the firing of every transition in T_1 requires at least one resource in R_1. But from (2) we know, in the marking M, all resources in R_1 are occupied by operations in ${}^{(p)}T_1$. So in any marking $M' \in RM(PN, M)$, there exists no any available resources in R_1, that is, T_1 is in deadlock in the marking M. ■

The following theorem presents the necessary conditions of deadlock in the systems.

Theorem 2: Given a system Petri net model $PN = (P \cup R, T, F, M_0)$ and a marking $M \in RM(PN, M_0)$, if $D(M) \neq \varnothing$, then there exist a subset of transitions $T_s \subseteq D(M)$ and a subset of resources R_s such that the following conditions hold

(1) For any transition $t \in T_s$, $M({}^{(p)}t)>0$;
(2) For any resource $r \in R_s$, $M({}^{(p)}T_s, r) = C_r$;
(3) For any transition $t \in T_s$, there exists a resource $r \in R_s$ such that $r \in {}^{(r)}t$;
(4) For any resource $r \in R_s$, there exist transitions $t_1, t_2 \in T_s$, such that $r \in R({}^{(p)}t_1)$, $r \in {}^{(r)}t_2$.

On the other hand, if T_s and R_s satisfy the conditions (1) ~ (4), by Lemma 1, T_s is in the deadlock in the marking M.

Proof: Let $t_1 \in D(M)$, then t is operation-enabled, but not resource-enabled, that is, $M({}^{(p)}t_1)>0$, and there exists a resource $r_1 \in {}^{(r)}t_1$, $M(r_1) = 0$. Hence there exist places $p_{11}, p_{12}, \ldots, p_{1s1} \in H(r_1)$, $M(p_{1i})>0$, $i = 1, 2, \ldots, s1$, such that $\sum_{i=1}^{s1} M(p_{1i}) = C_{r1}$. Let $p_{1i}^{\bullet} = t_{1i}$, $i = 1, 2, \ldots, s1$, $T_1 = \{t_{11}, t_{12}, \ldots, t_{1s1}\}$. Then $T_1 \subseteq D(M)$. For any transition $t_2 \in T_1$, similarly there exist a resource $r_2 \in {}^{(r)}t_2$, $M(r_2) = 0$, and places $p_{21}, p_{22}, \ldots, p_{2s2} \in H(r_2)$, such that $M(p_{2i}) > 0$, $i = 1,$

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on March 08,2026 at 05:17:56 UTC from IEEE Xplore. Restrictions apply.

2, …, s_2, $\sum_{i=1}^{s2} M(p_{2i}) = C_{r2}$. Let $p_{2i}^{\bullet}$ = t_{2i}, i = 1, 2, …, s_2, $T_2 = \{t_{21}, t_{22}, \ldots t_{2s2}\}$. Similarly $T_2 \subseteq D(M)$. Repeating the above analysis procedure, we can get a sequence of transitions t_1, t_2, … and a sequence of resources r_1, r_2, …. T and R are the finite set. So some transitions in t_1, t_2 … or some resources in r_1, r_2… must appear repeatedly. For simplicity, let t_1 be a first transition appearing repeatedly (the following proof is same if resources appear first repeatedly). In the following we use the recursive method to construct the set of transitions T_s and the set of resources R_s satisfying the conditions in Theorem 2. And let X be the set of transitions that are not analyzed, and at the beginning set $T_s = \varnothing$, $R_s = \varnothing$, $X = \{t_1\}$.

For any transition $t \in X \backslash T_s$, from $t \in D(M)$ we know there exists a resource $r \in {}^{(r)}t$ such that M(r) = 0. Hence there exist places $p_1, p_2, \ldots, p_k \in H(r)$ such that $M(p_i) > 0$, i = 1, 2, …, k, and $\sum_{i=1}^{k} M(p_i) = C_r$. Let $t_i = p_i^{\bullet}$, Y = {t_i, i = 1, 2, …, k}. Then $Y \subseteq D(M)$.

Let $T_s := T_s \cup \{t\}$, $R_s := R_s \cup \{r\}$, $X := X \cup Y$.

Repeat the above procedure till $X \backslash T_s = \varnothing$. By the above construction procedure of T_s and R_s, we can see that T_s and R_s satisfy the conditions (1)~(4) in Theorem 2.■

The condition (3) and (4) are important to the synthesis of Petri net controllers. Here a couple of a transition set T_s and a resource set R_s satisfying condition (3) and (4) is called a deadlock structure (DS), denoted by $<T_s, R_s>$. By definition, DS's are related to Petri net structures, not any markings of PN.

Theorem 2 implies that if the system is in deadlock in some marking $M \in RM(PN, M_0)$, then the system Petri net model PN contains some DS's. Hence we have the following necessary condition for guaranteeing the system is deadlock-free.

Theorem 3: Given a system Petri net model PN = ($P \cup R$, T, F, M_0), if PN does not contain any DS, then PN is deadlock-free. ■

But the following example demonstrates that the existence of DS's is only necessary, but not sufficient condition for leading the system to deadlock.

Example 1: Fig. 1 is the Petri net model of the system with two machines m_1 and m_2, and one robot r. The system

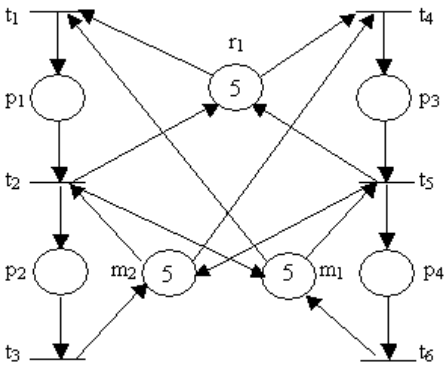

Fig. 1 A simple system Petri net model containing DS, but deadlock-free

can processes two kinds of parts: q_1 and q_2. Their process paths are m_1m_2 and m_2m_1, respectively and their first operations need the robot help. That is, $PR(q_1) = (\{m_1, r\}, \{m_2\})$ and $PR(q_2) = (\{m_2, r\}, \{m_1\})$. The capacities of three resources are all 5. Let $T_s = \{t_1, t_2\}$ and $Rs = \{m_1, m_2\}$, then $<T_s, R_s>$ is a DS. But because of existence of robot r, the system is deadlock-free. ■

IV. DESIGN OF PETRI NET CONTROLLER

In Section 3 we know that the existence of a DS $<T_s, R_s>$ is a necessary, but not sufficient condition for leading the system to deadlock. Only when the system reaches a making or a state M that satisfies the conditions in Theorem 2, that is, when all resources in R_s are occupied by the operations in ${}^{(p)}T_s$, $<T_s, R_s>$ leads the system to deadlock. So to avoid deadlock in the system, the controller must limits operations in ${}^{(p)}T_s$ to occupy the resources in R_s. In this paper we add some control places to enforce the system liveness. That is, for a given DS, add a control place and its relative arcs to the system Petri net model so that the controlled system is deadlock-free.

A. Designing Petri Net Controller Structure

For a DS $<T_s, R_s>$, we'll use a control place to limit the usage of resources in R_s by operations in ${}^{(p)}T_s$. So the structure design of the Petri net controller is actually to compute the arcs related with control places.

Definition 2: Given a system Petri net model PN = ($P \cup R$, T, F, M_0) and a DS $<T_s, R_s>$,

(1) A input path of $<T_s, R_s>$ is a directed path with nodes in T and P: $\pi = t_0p_1t_1p_2\ldots p_nt_n$, where only the end transition t_n is in T_s, $RE(p_i) \cap R_s \neq \varnothing$, i = 1, 2, …, n, and $RE({}^{(p)}t_0) \cap R_s = \varnothing$. Let $E(T_s, R_s)$ denote the set of the first transitions of all input paths of $<T_s, R_s>$.

(2) An output path of $<T_s, R_s>$ is a directed path with nodes in T and P: $\pi = t_0p_1t_1p_2\ldots p_nt_n$, where only the start transition t_0 is in T_s, $R(p_i) \cap R_s \neq \varnothing$, i = 1, 2, …, n, and $R(t_n^{(p)}) \cap R_s = \varnothing$.

Definition 3: Given a system Petri net model PN = ($P \cup R$, T, F, M_0) and a DS $<T_s, R_s>$, the subnet determined by $<T_s, R_s>$, denoted as $PN[T_s, R_s]$, is a subnet of PN spanned by the set of places $R_s \cup P_1$ and the set of transitions ${}^{\bullet}P_1 \cup P_1^{\bullet}$, where the set of places $P_1 = {}^{(p)}T_s \cup T_s^{(p)} \cup \{p \in P \mid p$ is in some input or output path of $<T_s, R_s>$ }.

Let M be a marking of PN, the limitation of M on $PN[T_s, R_s]$ is denoted by $M' = M|_{PN[Ts, Rs]}$. If in the marking M, $<T_s, R_s>$ satisfies the conditions in Theorem 2 and leads deadlock in the system, then in $PN[T_s, R_s]$, T_s is in deadlock in the marking M'. On the other hand, let M' be a marking of $PN[T_s, R_s]$, M a reachable marking of PN, and $M' = M|_{PN[Ts, Rs]}$. If in M', T_s is in deadlock in the marking M', then in PN, T_s is in deadlock in the marking M. Hence the Petri net controllers for avoiding deadlock in PN leaded by DS $<T_s, R_s>$ and for avoiding deadlock in $PN[T_s, R_s]$ are the same.

For a given DS $<T_s, R_s>$, let cp_s be a control place corresponding to it. Because the firing of any transition in

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on March 08,2026 at 05:17:56 UTC from IEEE Xplore. Restrictions apply.

$E(T_s, R_s)$ requires some resources in R_s, the Petri net controller should contain arcs from cp_s to every transition $t \in E(T_s, R_s)$. And the firing of any transition in $T_s \backslash {}^{\bullet}({}^{(p)}T_s)$ decreases the number of parts in ${}^{(p)}T_s$. After the firing of transitions in $T_s \backslash {}^{\bullet}({}^{(p)}T_s)$, the further processing of parts do not require any more resources in R_s. So the Petri net controller should contain arcs from every transition in $T_s \backslash {}^{\bullet}({}^{(p)}T_s)$ to cp_s. Hence the whole structure of Petri net controller can be defined as in the following.

Definition 4: Given a system Petri net model $PN = (P \cup R, T, F, M_0)$. For every DS $<T_s, R_s>$ in PN, define the substructure of the Petri net controller as following.

$$(\{cp_s\}, E(T_s, R_s) \cup (T_s \backslash {}^{\bullet}({}^{(p)}T_s)), F_s)$$

where $F_s = \{(cp_s, t) \mid t \in E(T_s, R_s)\} \cup \{(t, cp_s) \mid t \in T_s \backslash {}^{\bullet}({}^{(p)}T_s)\}$.

The whole structure of Petri net controller for the system is the union of substructures corresponding to all DS's in PN.

B. Computing Initial Markings of Petri Net Controllers

We compute now, recursively, the number of initial tokens in the control places, that is, according to the conclusion relation among all DS's, from inside to outside design the initial marking of the control places. Here DS $<T_2, R_2>$ concludes DS $<T_1, R_1>$ if $T_1 \subseteq T_2$ and $R_1 \subseteq R_2$.

Definition 5: Given a system Petri net model $PN = (P \cup R, T, F, M_0)$ and its DS $<T_s, R_s>$. Let cp_s be the control place corresponding to $<T_s, R_s>$. Then the initial number of tokens in cp_s is defined recursively as following.

(1) If $<T_s, R_s>$ does not contain any other DS, then the initial number of tokens in cp_s is

$$M_{c0}(cp_s) = DC(T_s, R_s) - 1$$

Where $DC(T_s, R_s) = \min\{ M(P_s) \mid M$ is a marking of $PN[T_s, R_s]$ and T_s is in deadlock in the marking $M\}$ and $P_s = {}^{(p)}T_s \cup \{p \in P \mid p$ is in some input path of $<T_s, R_s>\}$.

(2) Let $<T_i, R_i>$, $i = 1, 2, \ldots, k$, be all DS's contained in $<T_s, R_s>$ and cp_i be the control places corresponding to $<T_i, R_i>$. The initial marking of cp_i have been determined. Let $C_s = \{cp_i, i = 1, 2, \ldots, k\}$ and $PN[T_s, R_s, C_s]$ be $PN[T_s, R_s]$ controlled under C_s. Then the initial number of tokens in cp_s is

$$M_{c0}(cp_s) = DC(T_s, R_s, C_s) - 1$$

Where $DC(T_s, R_s, C_s) = \min\{ M(P_s) \mid M$ is a marking of $PN[T_s, R_s, C_s]$, T_s is in deadlock under the marking $M\}$, and $P_s = {}^{(p)}T_s \cup \{p \in P \mid p$ is in some input path of $<T_s, R_s>\}$.

Given a system Petri net model $PN = (P \cup R, T, F, M_0)$ and its Petri net controller C defined in Definition 4 and 5, we use CPN to denote the Petri net model of the controlled system, that is, PN controlled under C. Let ${}^{(c)}t$ and $t^{(c)}$ denote the sets of input control places and output control places of t, respectively. If for any control place $cp \in {}^{(c)}t$, $M(cp) \geq 1$, then t is called to be control-enabled. In CPN, only operation-, resource-, and control-enabled transitions can fire. For a given marking M of CPN, let D(CPN, M) denote the set of all transitions which are operation-enabled, but in deadlock in the marking M.

By the definition of the structure and initial marking of a Petri net controller, it is clear: if the presented Petri net controller can guarantee that the controlled system is deadlock-free, then it is an optimal Petri net controller with the considered Petri net controller structure.

C. Liveness of Controlled Systems

From the definition of the Petri net controller, we have its following property.

Property 1: Given a system Petri net model $PN = (P \cup R, T, F, M_0)$ and a DS $<T_s, R_s>$. Let $<T_i, R_i>$, $i = 1, 2, \ldots, k$, be all DS's contained in $<T_s, R_s>$, including $<T_s, R_s>$ itself. cp_i is the control place corresponding to $<T_i, R_i>$, $i = 1, 2, \ldots, k$. Let $C_s = \{cp_i, i = 1, 2, \ldots, k\}$ and $PN[T_s, R_s, C_s]$ be $PN[T_s, R_s]$ controlled under Cs. M is a marking of $PN[T_s, R_s, C_s]$. If $M(cp_s) = 0$, then by the definition of the initial marking of the Petri net controller, in $PN[T_s, R_s, C_s]$ there exists at least a transition $t \in T_s$ which is operation-, resource- and control-enabled in the marking M, that is, it can fire in the marking M. ■

The following theorem proves that the presented Petri net controller can avoid all deadlocks in the controlled systems.

Theorem 4: Given a system Petri net model $PN = (P \cup R, T, F, M_0)$. Let C be a Petri net controller as defined in Definition 4 and 5. Then the controlled system CPN is deadlock-free. Hence the controller C is an optimal Petri net controller with the considered Petri net controller structure.

Proof: We prove that for any reachable marking of CPN, $D(CPN, M) = \varnothing$.

If $D(CPN, M) \neq \varnothing$, then for any transition $t \in D(CPN, M)$, one of the following two cases must occur.

Case A: In the marking M, t is not resource-enabled. That is, there exists a resource $r \in {}^{(r)}t$, $M(r) = 0$, or

Case B: In the marking M, t is resource-enabled, but not control-enabled, that is, there exists a control place $cp_1 \in {}^{(c)}t$, $M(cp_1) = 0$.

If for any transition $t \in D(CPN, M)$, only Case A holds, then in the marking M, the Petri net controller C does not disable any transition in D(CPN, M). Hence by Theorem 2 there exists a DS $<T_s, R_s>$ such that for any resource $r \in R_s$, $M({}^{(p)}T_s, r) = C_r$. This contradicts to the definition of C. Hence there exist some transitions in D(CPN, M) such that Case B holds.

For a transition $t_1 \in D(CPN, M)$, if Case A holds, that is, there exists a resource $r_1 \in {}^{(r)}t_1$, $M(r_1) = 0$. Hence there exist transitions $t_{11}, t_{12}, \ldots, t_{1s1}$ such that $r_1 \in R({}^{(p)}t_{1i})$ and $M({}^{(p)}t_{1i}) > 0$, $i = 1, 2, \ldots, s1$, $\sum_{i=1}^{s1} M({}^{(p)}t_{1i}) = C_{r1}$. Again for transitions t_{1i}, $i = 1, 2, \ldots, s1$, Case A or Case B occurs.

For a transition $t_1 \in D(CPN, M)$, if Case B holds, that is, there exists a control place $cp_1 \in {}^{(c)}t_1$, $M(cp_1) = 0$. Let cp_1 correspond to a DS $<T_1, R_1>$. By Property 1, in the marking $M|_{PN[T1, R1, C1]}$, there exists some transition t in $PN[T_1, R_1, C_1]$ which is operation-, resource-, and control-enabled. Let $t_{21}, t_{22}, \ldots, t_{2s2}$ be all this kind of transitions. Then in CPN,

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on March 08,2026 at 05:17:56 UTC from IEEE Xplore. Restrictions apply.

$t_{2i} \in D(CPN, M)$ and for t_{2i}, one of the following two Cases occurs.

Case C: t_{2i} is not resource-enabled, that is, there exists a resource place $r_{2i} \in {}^{(r)}t_{2i}$, $M(r_{2i}) = 0$. This case is similar to Case A. But here there exist transitions $t_{31}, t_{32}, \ldots, t_{3s3}$ which are not all in T_1 such that $r_{2i} \in R({}^{(p)}t_{3i})$, i = 1, 2, …, s3, $\sum_{j=1}^{s3} M({}^{(p)}t_{3j}) = C_{r2i}$. For every transition t_{3i}, again one of Case A and Case B occurs.

Case D: t_{2i} is resource-enabled, but not control-enabled in the marking M, that is, there exists a control place $cp_3 \in {}^{(c)}t$, $M(cp_3) = 0$. Actually this case is similar to Case B. But here the DS $<T_3, R_3>$ corresponding to cp_3 is not included in $<T_1, R_1>$.

Repeat the above analysis procedure. Because PN is finite, some transitions must occur repeatedly. For simplicity, suppose that t_1 is such kind transition. And Let T_s and R_s be the sets of all transitions and all resource places that occur repeatedly, respectively. That is, T_s contains all transitions in every case and transitions in every DS appearing in analysis procedure, R_s contains all resources corresponding to every transition. For example, in the analysis, in Case A transitions t_1 and $t_{11}, t_{12}, \ldots, t_{1s1}$, resource r_1; in Case B, the transition set T_1 and resource set R_1; in Case C, transitions $t_{31}, t_{32}, \ldots, t_{3s3}$, and resources $r_{31}, r_{32}, \ldots, r_{3s3}$; in Case D, transition set T_3 and resource set R_3 are all contained in T_s and R_s, respectively. Here we only need to prove that T_s and R_s form a DS. If transitions or resources are in some DS, they already satisfy the conditions for DS. So we only check the transitions or resources appearing in Case A or Case C. Let $t \in T_s$. For t Case A occurs, that is, there exists a resource place $r \in {}^{(r)}t$, $M(r) = 0$. Then there exist some operation-enabled transitions $t_1, t_2, \ldots, t_s$ such that $r \in R({}^{(p)}t_i)$, i = 1, 2, …, s, $\sum_{i=1}^{s} M({}^{(p)}t_i) = C_r$. Hence $t_i \in T_s$, i = 1, 2, …, s, $r \in R_s$, that is, $r \in {}^{(r)}t \cap R_s \neq \varnothing$, condition (3) of a DS holds. And for the resource r, because $t \in T_s$, $r \in R({}^{(p)}t)$, condition (4) of a DS holds. For transitions and resources in Case C, the proof is similar. So $<T_s, R_s>$ is a DS.

By the above procedure we know that $<T_s, R_s>$ is in deadlock in the marking $M|_{PN[Ts, Rs, Cs]}$ of $PN[T_s, R_s, C_s]$. But from the definition of the controller C, in $PN[T_s, R_s, C_s]$ there exists at least one transition $t \in T_s$ which can fire. This contradiction implies that $D(CPN, M) = \varnothing$.■

V. CONCLUSIONS.

This paper discusses the deadlock problems arising in automated manufacturing systems with multiple resource service. We use Petri net to model the system and characterize the necessary and sufficient conditions with Petri net substructures: DS. For each DS, by adding a control place to system Petri net model and limiting the usage of resources in DS, deadlock in the controlled system can be prevented. The presented deadlock avoidance Petri net controllers are optimal. But some control places are not necessary for avoiding deadlock, that is, the controller is not minimal in the structure. How to get a minimal structure and maximal permissive deadlock avoidance Petri net should be further investigated.

REFERENCES

[1]. Z. A. Banaszak and B. H. Krogh, "Deadlock avoidance in flexible manufacturing systems with concurrently competing process flows," IEEE Trans. Robot. Automat., vol. 6, pp. 724–734, Dec. 1990

[2]. J. Ezpeleta, J. M. Colom, and J. Martinez, "A Petri net based deadlock prevention policy for flexible manufacturing system," IEEE Trans. Robot. Automat., vol. 11, pp. 173–184, Apr. 1995.

[3]. M. P. Fanti, "Deadlock Resolution Strategy for Automated Manufacturing Systems Including Conjunctive Resource Service", IEEE Trans. Syst., Man, Cybern. A, vol. 34, pp.80–91, Jan. 2004.

[4]. M. P. Fantic, G. Maione, and B, Turchiano, "Design of Supervisor to Avoid Deadlock in Flexible Assembly Systems," Int. J. Flexible Manuf. Syst., vol.14, pp.157-175, 2002

[5]. J. park and S. A. Reveliotis, "Deadlock Avoidance in Sequential Resource Allocation Sytems with Multiple Resource Acquisition and Flexible Routing," IEEE Trans. Automat. Contr., vol.46, pp.1572-1583, Oct. 2001.

[6]. Z. W. Li and M. C. Zhou, "Elementary Siphons of Petri Nets and Their Application to Deadlock Prevention in Flexible Manufacturing Systems", IEEE Trans. Syst., Man, Cybern. A, vol. 34, pp.38–51, Jan. 2004.

[7]. E. Roszkowska, "Supervisory Control for deadlock Avoidance in Compound Process", IEEE Trans. Syst., Man, Cybern. A, vol. 34, no. 1, pp. 52–64, Jan. 2004.

[8]. M. C. Zhou and F. DiCesare, Petri Net Synthesis for Discrete Event Control of Manufacturing Systems, Norwell, MA: Kluwer, 1993.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on March 08,2026 at 05:17:56 UTC from IEEE Xplore. Restrictions apply.